

作业一：文本分类实现

基于词袋、词向量与预训练语言模型的比较

姓名：杨桑多杰 学号：2313943

2026 年 10 月 9 日

摘要

本实验在 NYT 新闻分类数据上比较了三类文本表示方法：词袋、静态词向量与预训练语言模型。数据按 80%/10%/10% 划分，所有模型共用同一份划分，结果统一在测试集上以 Accuracy 与 Macro-F1 报告。实际完成了六组实验：二值词袋与词频词袋两组词袋基线，GloVe 100 维、AG News 词向量、NYT 词向量三组词向量基线，以及 `bert-base-uncased` 的微调。

结果里最好的一组是用 NYT 训练集自己训练的 100 维 Word2Vec，Accuracy 0.9922、Macro-F1 0.9792，两项都超过了词袋基线（词频词袋 0.9896 / 0.9743）。预训练 GloVe 与 AG News 上训练的 Word2Vec 都在 0.9852 / 0.96 附近，反而不如词袋。BERT 在作业规定的 `max_length=64`、3 epochs 条件下只拿到 0.9818 / 0.9612，六组里排在最后，原因主要在于 NYT 文本太长（训练集平均 830 个词元），99.9% 的文档都被截断，模型实际只能看到开头一小段。考虑到 sports 类占了 75%，除 Accuracy 外，Macro-F1 也是必须看的指标。

1 实验目标与任务

这次作业的目的是把几种文本表示方法都动手实现一遍，比较它们在同一个分类任务上的实际差别。具体分三部分：

1. Task 1: 用 Binary Bag of Words 和 Word Frequency 两种词袋表示文档，配 Logistic Regression 分类器；
2. Task 2: 用 100 维词向量表示文档，来源分别是预训练 GloVe、在 AG News 上训练的 Word2Vec、在 NYT 上训练的 Word2Vec，文档向量取词向量平均，分类器仍是 Logistic Regression；
3. Task 3: 直接微调 `google-bert/bert-base-uncased`。

所有实验都在同一个 NYT 测试集上报告 Accuracy 与 Macro-F1。

2 数据集与预处理

2.1 数据集概况

HW-1/nyt.csv 有 11,519 条带标签新闻，两列分别是 `text` 和 `label`。类别分布很不均衡：sports 8,639 条、politics 1,451 条、business 1,429 条，sports 一类就占了 75%。这种情况下只看 Accuracy 容易被多数类掩盖问题，所以 Macro-F1 必须一并报告。

HW-1/ag.csv 有 90,000 条新闻文本，没有标签，只用来给 Task 2 训练一个领域 Word2Vec。

2.2 数据划分与文本预处理

先把 NYT 整体打乱，再按 80%/10%/10% 切成训练、验证、测试三份，实际是 9,215 / 1,152 / 1,152 条。划分用种子 42 加分层抽样，保证三份的类别比例和原始数据基本一致。这里特意把划分结果存成了 CSV (code/split/)，后面所有实验都读同一份——不然每次重新划分，各方法之间就没法横向比较了。

传统模型用的是同一套分词规则：转小写，用 `nltk.word_tokenize` 切词，只保留长度不小于 2 的纯字母词元。词表只从训练集统计，避免测试集的词混进来。如果 `nltk` 的 `punkt` 数据没下下来，代码会自动退回正则分词，结果基本不受影响。

3 方法设计

3.1 Task 1: 词袋模型

训练集词表大小实测 $|V| = 57,535$ ，每篇文档就表示成这个维度的向量。

- **Binary Bag of Words:** 词出现记 1，不出现记 0，完全不管出现几次；
- **Word Frequency:** $x_i = c_i$ ，记的是每个词在当前文档里的实际次数。

分类器统一用 One-vs-Rest 的 Logistic Regression (L2 正则、`liblinear` 求解、`max_iter=500`)。正则强度 C 在 $\{0.1, 1, 10\}$ 里挑，按验证集 Macro-F1 选最好的那个，测试集只在最后评估一次。

3.2 Task 2: 静态词向量

三种词向量都是 100 维，文档表示统一取文档内所有命中词向量的平均：

$$\mathbf{d} = \frac{1}{n} \sum_{i=1}^n \mathbf{e}(w_i), \quad (1)$$

其中 n 是能命中词向量的词数；如果一篇文档一个词都没命中，就取全零向量（实际数据里基本不会出现）。

- **Pre-trained GloVe:** 直接用公开的 `glove.6B.100d.txt`，40 万词条、100 维；
- **Word2Vec on AG News:** 用 `gensim` 在 90,000 条 AG News 文本上现训一个，Skip-gram，窗口 5，`min_count=2`，负采样 5，跑 10 个 epoch；
- **Word2Vec on NYT:** 参数相同，语料换成 NYT 训练集（只用训练集，免得词向量提前见过测试文本）。

拿到文档向量后先做标准化（`StandardScaler` 只在训练集上拟合），再与 Task 1 相同的 Logistic Regression， C 的候选扩到 $\{0.1, 1, 10, 100\}$ 。

3.3 Task 3: BERT 微调

使用 `google-bert/bert-base-uncased` 预训练模型及其 tokenizer，作业规定的三个设置原样照做：

- Pre-trained model: `bert-base-uncased`;
- Maximum sequence length: 64 (超出截断、不足补齐);
- Number of epochs: 3。

在预训练模型后面接一个 3 分类线性头。优化器用 AdamW，学习率 2×10^{-5} ，batch size 32，种子 42，配 10% 的 warmup 和线性衰减。每个 epoch 结束在验证集上看一次，测试集只在最后跑一次。训练在一块 NVIDIA 显卡上完成。

4 实现与复现

完整代码都在 `code/` 目录下：

- `common.py`: 共用的数据加载、80/10/10 划分、分词与评价指标;
- `task1_bow.py`: Task 1 的两种词袋表示;
- `task2_glove.py` 与 `task2_word2vec.py`: Task 2 的三组词向量实验;
- `task3_bert.py`: Task 3 的 BERT 微调;
- `run_all.py`: 一键跑全部流程; `make_figure.py`: 生成结果对比图;
- `analyze_truncation.py`: 统计 BERT 输入的截断情况 (正文中「833 个词元、99.9% 被截断」即出自这里); `download.py` 与 `download_bert_files.py`: 带断点续传的下载工具。

运行环境为 Python 3.12，依赖见 `code/requirements.txt` (pandas、numpy、scikit-learn、nltk、gensim、torch、transformers; 结果对比图另需 matplotlib)。GloVe 文件 `glove.6B.100d.txt` 置于 `HW-1/` 下; BERT 权重可放在 `code/artifacts/bert-base-uncased/`，或由脚本从 Hugging Face 自动下载 (国内网络可先设置镜像 `HF_ENDPOINT=https://hf-mirror.com`)。一键运行命令：

Listing 1: 一键运行

```
cd code && python run_all.py
```

每步实验的测试集指标会自动写进 `code/results/*.json`，下面表格里的数字就是从这里来的。

5 实验结果

5.1 Task 1: 词袋模型

表1为两种词袋表示在测试集上的结果，词表统一为 $|V| = 57,535$ 维。两种方法都做到 0.99 上下，Word Frequency 略好一点，但两者差距很小。

表 1: Task 1: 词袋模型在 NYT 测试集上的结果 (实测)

表示方法	最佳 C	Accuracy	Macro-F1
Binary Bag of Words	0.1	0.9887	0.9727
Word Frequency	0.1	0.9896	0.9743

5.2 Task 2: 静态词向量

表2为三种 100 维词向量平均池化 + Logistic Regression 的结果。NYT 自己训的词向量表现最好 (Accuracy 0.9922、Macro-F1 0.9792)，也是六组实验里的最高分；预训练 GloVe 与 AG News 训的 Word2Vec 都在 0.9852 / 0.96 附近，反而低于两种词袋基线。

表 2: Task 2: 词向量表示在 NYT 测试集上的结果 (实测)

方法	最佳 C	Accuracy	Macro-F1
GloVe 100d + LR	0.1	0.9852	0.9646
Word2Vec (AG News) 100d + LR	1	0.9852	0.9637
Word2Vec (NYT) 100d + LR	1	0.9922	0.9792

5.3 Task 3: BERT 微调

表3为 BERT 微调 (`max_length=64`, 3 epochs) 在 NYT 测试集上的结果。

表 3: Task 3: BERT 微调在 NYT 测试集上的结果 (实测)

方法	Accuracy	Macro-F1
BERT-base-uncased fine-tuning	0.9818	0.9612

5.4 总体对比

表4把所有方法放在一起看，图1是对应的柱状图。

6 分析与讨论

词袋内部比较。两种词袋的差距很小：Macro-F1 0.9727 对 0.9743，Accuracy 0.9887 对 0.9896，基本可以看作持平。想一下也合理——新闻这种主题性很强的文本，「某个词有没有出现」本身就已经包含大部分判别信息了，再记一个次数并没有增加什么。

词向量。三组词向量里只有 NYT 自己训的那组超过了词袋：Accuracy 0.9922 对 0.9896，Macro-F1 0.9792 对 0.9743，GloVe 与 AG News 词向量则都没做到。差别主要还是出在语料上：NYT 词向量和目标数据同源，共现统计自然更贴合任务本身；GloVe 词表虽然大得多，但来自通用语料，和新闻分类这个场景还是有距离。AG News 与 GloVe 几乎持平，说明领域接近程度比词表规模更关键。

表 4: 全部方法在 NYT 测试集上的结果汇总

方法	Accuracy	Macro-F1
Binary Bag of Words + LR	0.9887	0.9727
Word Frequency + LR	0.9896	0.9743
GloVe 100d + LR	0.9852	0.9646
Word2Vec (AG News) + LR	0.9852	0.9637
Word2Vec (NYT) + LR	0.9922	0.9792
BERT-base-uncased (微调)	0.9818	0.9612

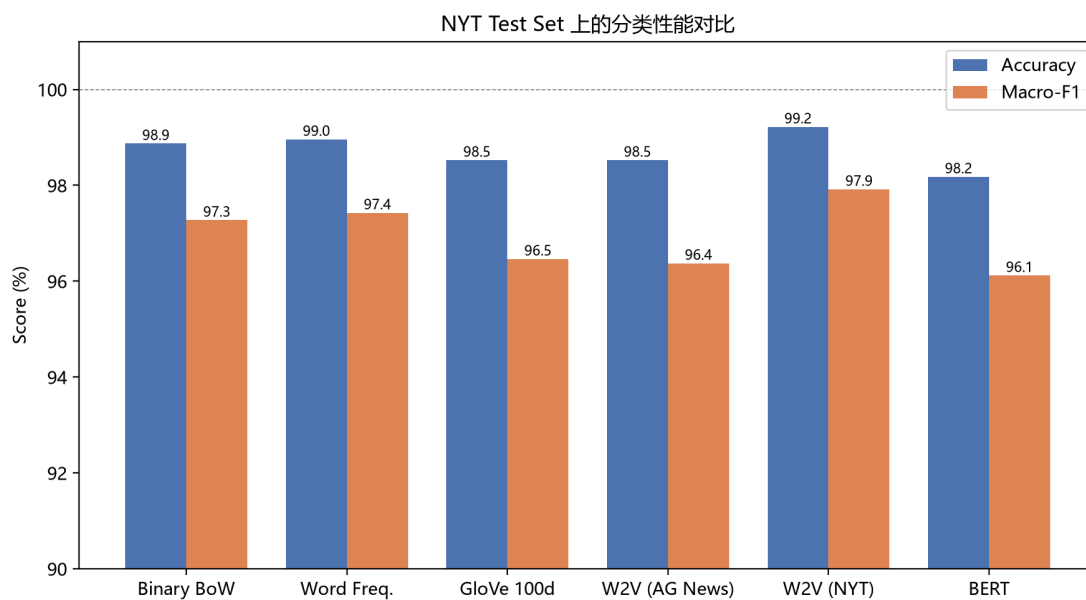


图 1: NYT 测试集上各方法的 Accuracy 与 Macro-F1 对比

另外，平均池化本身是有损的：一篇文章里所有词等权求和，「game 出现 10 次」和「出现 1 次」在池化之后完全没区别，词频和关键实体的信息就这么丢了。NYT 词向量能在这种条件下仍然胜出，说明词向量本身训得好不好，比用哪种池化方式更起作用。

BERT。这部分是本次实验里最值得说一说的。BERT 拿到 0.9818 / 0.9612，六组里排在最后，不但没赢过词向量，连最简单的词袋也没赢过。分析下来原因主要有两个。第一是截断：NYT 的新闻非常长，训练集平均 833 个词元、中位数 871，而作业把最大长度固定成 64，结果 99.9% 的文档都被截掉，模型实际只能看到开头 60 来个词元，也就是标题加前两行，判别信息大部分躺在后面的正文里。第二是过拟合的迹象：3 个 epoch 里训练损失一路降到 0.023，但验证集 Macro-F1 在第 2 个 epoch 就到顶 (0.9484)，第 3 个 epoch 反而回落到 0.9402。也就是说，这个设置下模型先撞上的是输入长度与训练轮数的天花板，而不是它自身能力的上限。如果放开序列长度、多训几轮，结果应该会不一样——这也提醒我，预训练模型并不是无条件更强的，用之前得先看清楚任务和超参的限制卡在哪里。

类别不均衡。数据里 75% 是 sports，Accuracy 基本被这个大类主导。所有方法在 Macro-F1 上的排名与 Accuracy 一致，但差距被放大了：比如 NYT 词向量与 Binary BoW 的 Accuracy 只差 0.0035，Macro-F1 却差了 0.0065。这正说明 Macro-F1 对每一类等权平均，能更真实地反映 politics、business 这两个少数类被识别得怎么样。

7 结论

按作业要求，在统一的数据划分与评价标准下把三类文本表示方法都做了一遍，结论大致如下：

1. 词袋 + Logistic Regression 是这类主题鲜明新闻分类上的强基线，词频词袋拿到 Accuracy 0.9896、Macro-F1 0.9743，两种表示之间几乎没有差别；
2. 三组词向量里，在目标数据上自己训的 Word2Vec 表现最好 (0.9922/0.9792)，也是全部实验的最高分；通用的 GloVe 与 AG News 词向量都低于词袋，说明训练语料与目标分布是否一致，比词表有多大更重要；
3. BERT 微调在 `max_length=64` 与 3 epochs 的固定设置下只做到 0.9818/0.9612，六组里排在最后，主要原因是长文本被强截断，叠加后期过拟合；给足输入长度和训练轮数，预训练模型应该还能再往上走。